
Laboratory Guide

Lesson 02 — Sensors, Interrupts and Visual Programming

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva

ESTGA, University of Aveiro

Estimated duration: 2 hours

Contents

Context	2
The two modes of operation used	2
Material and Prior Preparation	2
Learning Outcomes	3
Session Plan	4
Initial Procedure: StandardFirmata	5
1 Controlling a Digital Output from Node-RED	5
2 Reading a Digital Input and Responding to Events	7
3 Acquiring an Analogue Quantity	9
4 Digital Sensor with Its Own Protocol (DHT11)	11
5 Ultrasonic Distance Sensor (HC-SR04)	13
6 Interrupts at Microcontroller Level	15
7 Integrated Display Panel	18
References	18

Context

This guide puts into practice, in the laboratory, the concepts presented in the lecture component of Lessons 01 and 02: the reference architecture of IoT systems, analogue-to-digital conversion and the mechanisms for responding to events.

The approach adopted favours **visual programming tools** (*low-code*). Wherever the technology allows, the application logic is built in Node-RED by connecting functional blocks, without writing code. Source code is used only in the cases where the sensors rely on timed protocols that the generic firmware does not support; in those cases the *sketches* are supplied ready to load, and it is not for the student to write them.

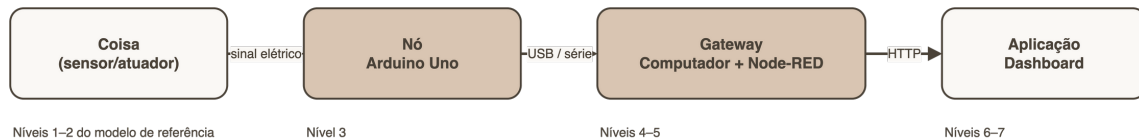
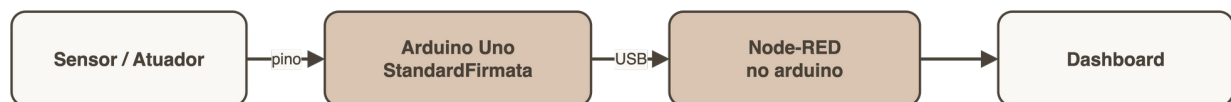


Figure 1: Position of the exercises in the IoT chain and in the reference model of Lesson 01

The two modes of operation used

Two distinct modes of articulation between the Arduino and Node-RED are used during the session, and the distinction is essential in order to understand the exercises.

Modo A — Sem código no Arduino (StandardFirmata)



Modo B — Sketch dedicado + porta série



Figure 2: Mode A (Firmata, no code) and Mode B (dedicated sketch with serial communication)

Mode A — Firmata

The Arduino runs the generic **StandardFirmata** firmware, loaded once at the start of the session. This firmware exposes the board pins to the computer, allowing Node-RED to read and write them directly. No code is written. It applies to exercises 1, 2 and 3.

Mode B — Dedicated *sketch* with serial communication

The DHT11 and HC-SR04 sensors use protocols with strict timing requirements, not supported by **StandardFirmata**. In these cases a specific *sketch* is loaded, supplied with the guide, which performs the reading and sends the result over the serial port. The processing and display of the data continue to be carried out in Node-RED. It applies to exercises 4 and 5.

Material and Prior Preparation

The laboratory sessions take place on the students' personal computers. Software installation must therefore be completed before the start of the class.

Equipment (provided in the laboratory)

- Arduino Uno and its USB cable
- *Breadboard* and connecting wires
- Push button and 10 k Ω resistor
- Soil moisture sensor (resistive hygrometer)
- DHT11 sensor (temperature and humidity)
- HC-SR04 sensor (ultrasonic distance)

Software to install beforehand

1. **Arduino IDE**, from arduino.cc/en/software. It is used exclusively to load firmware onto the board.
2. **Node.js**, LTS version, from nodejs.org.
3. **Node-RED**, through the command `npm install -g --unsafe-perm node-red`.
4. **Additional Node-RED packages**, installed under `≡ > Manage palette > Install:` `node-red-node-arduino`, `node-red-node-serialport` and `@flowfuse/node-red-dashboard`.
5. **Arduino libraries**, installed under `Tools > Manage Libraries`, searching by name: `TimerOne`, used in the software interrupt exercise, and `DHTlib`, by Rob Tillaart, used to read the DHT11 sensor. Should either not appear in the manager, it can be obtained from the repository given in the references and imported under `Sketch > Include Library > Add .ZIP Library`.
6. On Windows systems, and should the board not be recognised, the CH340 USB-serial driver must be installed.

Note on the libraries

Neither library is distributed with the Arduino IDE or with this guide. They are third-party projects, freely usable, and must be installed by the student before the session. Installation takes under a minute per library and requires no registration.

Checking that it works

Run `node-red` in a terminal and open `http://127.0.0.1:1880`. The flow editor should be displayed. The node palette, on the left, should include the `arduino` and `dashboard` sections and the `serial in` node.

Learning Outcomes

By the end of the session, the student should be able to:

- Establish the physical connection between sensors and actuators and a microcontroller platform, correctly identifying power, ground and signal lines;
- Build flows for acquiring, processing and displaying data in Node-RED, without resorting to textual programming;
- Distinguish analogue from digital sensors and relate the reading obtained to the resolution of

the analogue-to-digital converter;

- Distinguish the paradigms of periodic polling and of interrupt, identifying how each manifests at the level of the visual flow and at the level of the microcontroller;
- Integrate multiple data sources into a single display panel.

Session Plan

No.	Exercise	Mode	Duration
1	Controlling a digital output from Node-RED	A	15 min
2	Reading a digital input and responding to events	A	20 min
3	Acquiring an analogue quantity (soil moisture)	A	15 min
4	Digital sensor with its own protocol (DHT11)	B	20 min
5	Ultrasonic distance sensor (HC-SR04)	B	20 min
6	Interrupts at microcontroller level	—	20 min
7	Integrated display panel	A + B	10 min

Organisation of the work

Groups of two students are recommended, preferably with different educational backgrounds. It is suggested that circuit assembly and flow construction be alternated between the members of the group halfway through the session, so as to ensure that both come into contact with the two dimensions of the work.

Initial Procedure: Loading StandardFirmata

This procedure is carried out once and enables exercises 1 to 3.

1. Connect the Arduino Uno to the computer through the USB cable.
2. In the Arduino IDE, select **Tools > Board > Arduino Uno** and, under **Tools > Port**, the port corresponding to the board. **Record the designation of this port**, required in all the exercises that follow.
3. Open **File > Examples > Firmata > StandardFirmata**.
4. Load the *sketch* onto the board through **Sketch > Upload**.
5. Close the Arduino IDE, or at least the *Serial Monitor*. The serial port can only be used by one application at a time, and it will be required by Node-RED.

Note

From this point until exercise 4, no other program is loaded onto the board. All the logic is defined in Node-RED.

1. Controlling a Digital Output from Node-RED

Mode A — Firmata / 15 min

Objective

Drive the LED built into the board, associated with pin D13, through a switch displayed on a web panel, without writing code.

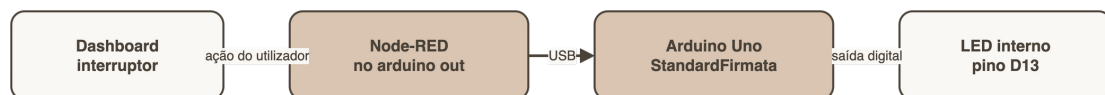


Figure 3: Architecture of exercise 1

Connections

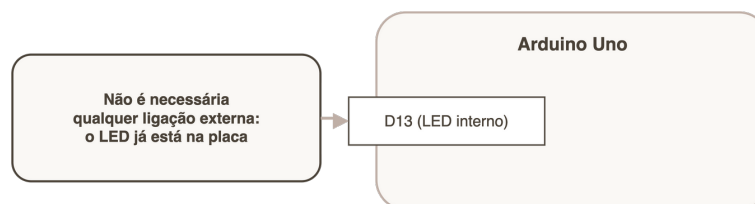


Figure 4: Connections for exercise 1

Building the flow

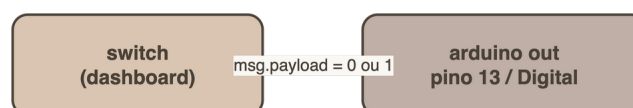


Figure 5: Node-RED flow for exercise 1

1. Drag a **ui-switch** node from the *dashboard* section onto the workspace.
2. Open the node with a double click. The **Group** field may be left empty: on the first **Deploy**, Dashboard automatically creates the page and group required.
3. In the same dialog, find the **On Payload** and **Off Payload** rows. Each has a small type selector on its left. Open that selector, choose **number**, and enter 1 on the first row and 0 on the second.
4. Drag an **arduino out** node from the *arduino* section.
5. Edit the node: in the **Arduino Port** field give the port recorded in the initial procedure; in **Pin** give 13; in **Type** select **Digital (0/1)**.
6. Connect the output of the **ui-switch** node to the input of the **arduino out** node.
7. Press **Deploy**.

If the fields are not where described

There are two versions of the Node-RED dashboard, with different nodes and addresses. This guide uses version 2, whose package is `@flowfuse/node-red-dashboard` and whose nodes all carry the `ui-` prefix. Version 1, `node-red-dashboard`, was deprecated in June 2024, has nodes without the prefix and serves the panel at `/ui` rather than `/dashboard`.

If the palette shows a node named **switch** in the *dashboard* section, rather than **ui-switch**, the old version is installed. In that case, uninstall `node-red-dashboard` under **Manage palette** and install `@flowfuse/node-red-dashboard`.

Verification

Open `http://127.0.0.1:1880/dashboard`. Operating the switch should turn the board LED on and off. Confirm also that the **arduino out** node shows the **connected** state in the editor.

Further work

Replace the **ui-switch** node with an **inject** node configured to repeat every second, followed by a **function** node that alternates the value sent. This yields, with no code on the board, the blinking behaviour traditionally programmed with `delay()`.

2. Reading a Digital Input and Responding to Events

Mode A — Firmata / 20 min

Objective

Detect the actuation of a push button and drive the LED according to its state, observing that the flow is executed only when a change occurs on the pin.

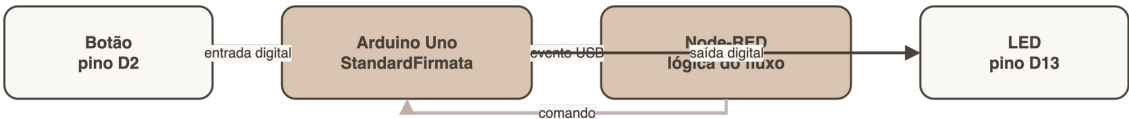


Figure 6: Architecture of exercise 2

Connections

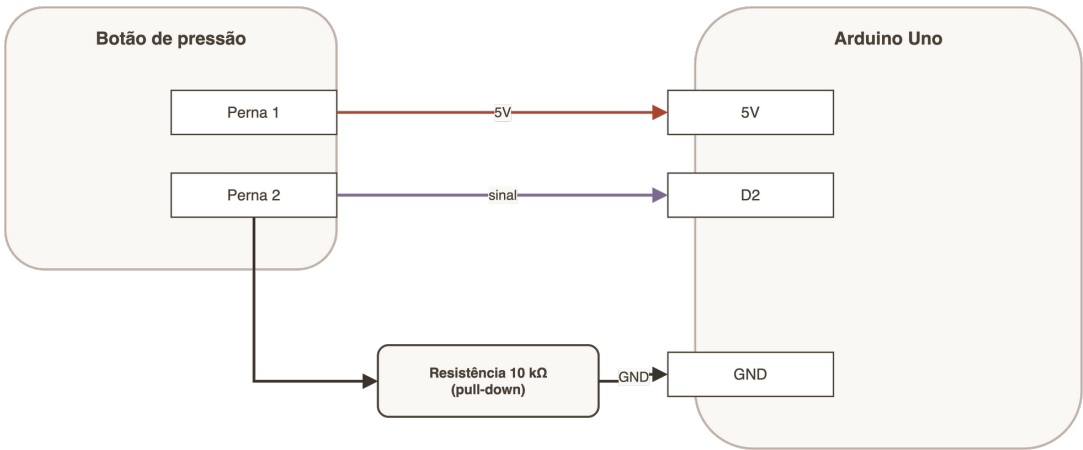


Figure 7: Connections for exercise 2

Button	Arduino	Function
Leg 1	5V	Power supply
Leg 2	D2	Signal line
Leg 2	GND, through the 10 kΩ resistor	<i>Pull-down</i> : it ensures logic level 0 with the button at rest

Assembly

Connections must be made with the board disconnected from the USB cable. The *pull-down* resistor is indispensable: without it the input is left floating and the value read fluctuates unpredictably.

Building the flow

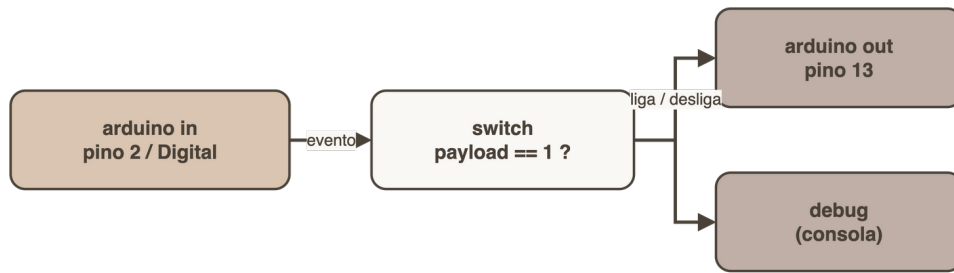


Figure 8: Node-RED flow for exercise 2

1. Drag an **arduino in** node. Configure the port, pin 2 and type **Digital** (0/1).
2. Drag a **switch** node from the *function* section. Set the rule `== 1` on the first output and `otherwise` on the second.
3. Drag an **arduino out** node, configured for pin 13, type **Digital** (0/1).
4. Drag a **debug** node and connect it also to the output of the **arduino in** node, so as to observe the messages in the side console.
5. Make the connections and press **Deploy**.

Verification

On pressing the button the LED should light; on releasing it, it should go out. In the *debug* console, messages are observed only at the instants of transition, and not continuously.

Conceptual observation

The **arduino in** node is not executed cyclically by the user: it emits a message when the pin value changes. At the level of the visual flow, this is the same event-driven principle that, at the microcontroller level, is realised by an interrupt service routine. Exercise 6 establishes this correspondence.

3. Acquiring an Analogue Quantity

Mode A — Firmata / 15 min

Objective

Acquire the soil moisture sensor reading through the Arduino analogue-to-digital converter and display it graphically, relating the value obtained to the 10-bit resolution discussed in the lecture.

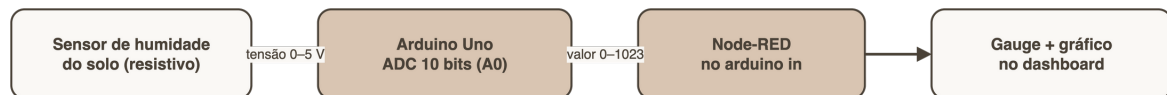


Figure 9: Architecture of exercise 3

Connections

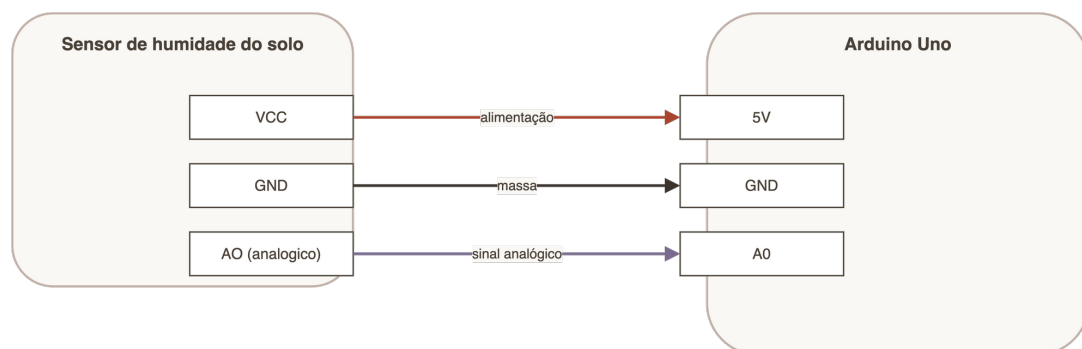


Figure 10: Connections for exercise 3

Sensor	Arduino	Function
VCC	5V	Power supply
GND	GND	Ground
AO	A0	Analogue output of the sensor

Building the flow

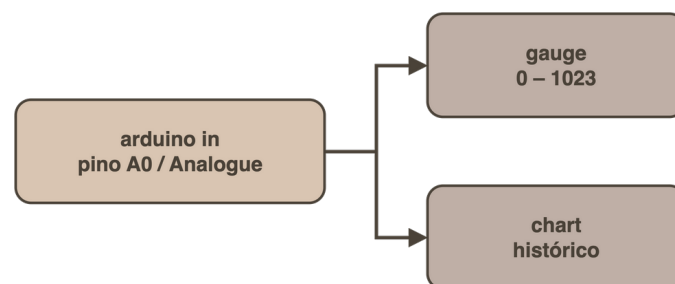


Figure 11: Node-RED flow for exercise 3

1. Drag an `arduino in` node. Configure the port, pin A0 and type `Analogue (0-1023)`.

2. Drag a **ui-gauge** node from the *dashboard* section. Set the range from 0 to 1023 and label it *Soil moisture*.
3. Drag a **ui-chart** node from the same section, keeping the default configuration.
4. Connect both to the output of the **arduino_in** node and press **Deploy**.

Verification

On the panel, the value displayed should vary according to the degree of moisture to which the sensor prongs are subjected. Record the value obtained with the sensor in air and with the prongs moistened.

Relation to the lecture

The value displayed lies between 0 and 1023 because it results from a conversion with 10-bit resolution ($2^{10} = 1024$ discrete levels), applied to a voltage between 0 V and the 5 V reference voltage.

Further work

Insert a **range** node between the acquisition and the **ui-gauge**, converting the 0–1023 scale into a percentage from 0 to 100. This is a scaling operation carried out entirely by configuration, without programming.

4. Digital Sensor with Its Own Protocol (DHT11)

Mode B — supplied sketch / 20 min

Objective

Acquire temperature and relative humidity from the DHT11 sensor and display them on two separate indicators, understanding why this sensor requires dedicated firmware.



Figure 12: Architecture of exercise 4

Connections

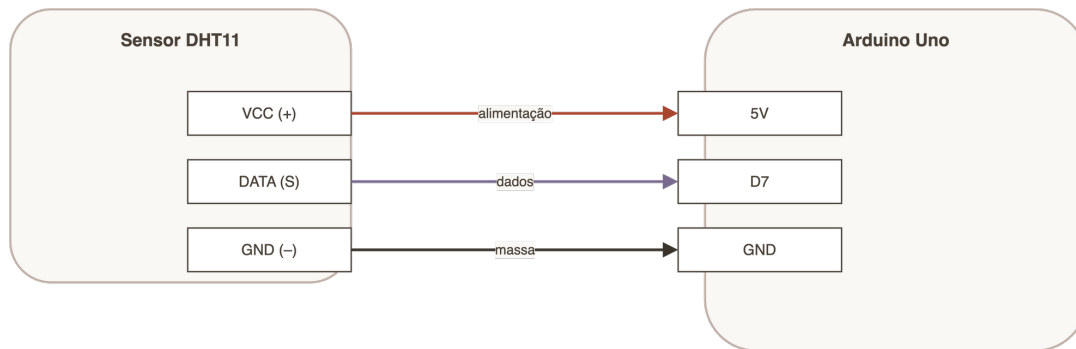


Figure 13: Connections for exercise 4

DHT11	Arduino	Function
VCC (+)	5V	Power supply
DATA (S)	D7	Bidirectional data line
GND (-)	GND	Ground

Loading the firmware

The DHT11 transmits data over a single line, using a time-based encoding with microsecond resolution. This protocol is not supported by `StandardFirmata`, so a dedicated *sketch* must be loaded.

1. Confirm that the `DHTlib` library is installed, as indicated in the prior preparation section.
2. Open the file `dht11_serie.ino`, supplied with this guide, and load it onto the board.
3. Close the *Serial Monitor* before returning to Node-RED.

```
#include <dht.h>
dht DHT;
#define DHT11_PIN 7

void setup() { Serial.begin(9600); }
```

```
void loop() {
  DHT.read11(DHT11_PIN);
  Serial.print(DHT.temperature);
  Serial.print(",");
  Serial.println(DHT.humidity);
  delay(2000);
}
```

Each line sent has the format `temperature,humidity`, suitable for later processing in Node-RED.

Building the flow

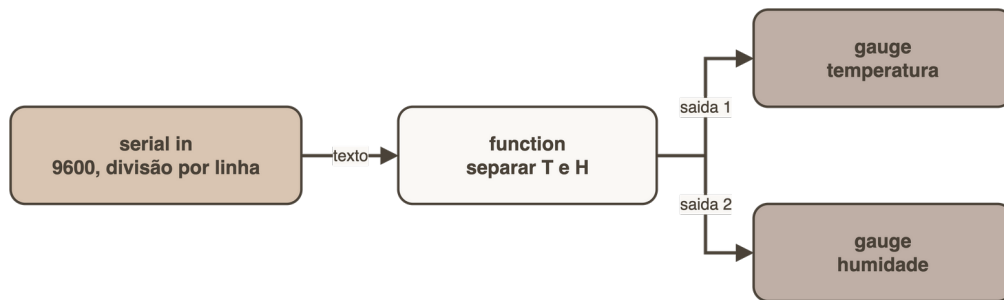


Figure 14: Node-RED flow for exercise 4

1. Drag a **serial in** node. Configure the port recorded earlier, the rate 9600 and splitting of the input on the end-of-line character.
2. Drag a **function** node with two outputs, containing:

```
var v = msg.payload.split(",");
return [ { payload: parseFloat(v[0]) },
        { payload: parseFloat(v[1]) } ];
```

3. Connect the first output to a **ui-gauge** node labelled *Temperature*, with range 0 to 50.
4. Connect the second output to a **ui-gauge** node labelled *Relative humidity*, with range 0 to 100.
5. Press Deploy.

Verification

The two indicators should update every two seconds. Bringing a hand close to the sensor should produce a noticeable variation in both quantities.

5. Ultrasonic Distance Sensor (HC-SR04)

Mode B — supplied sketch | 20 min

Objective

Determine the distance to an obstacle by measuring the propagation time of an ultrasonic pulse and display its evolution over time.

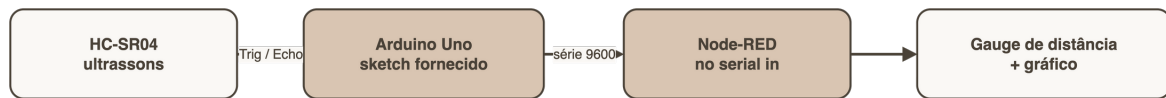


Figure 15: Architecture of exercise 5

Connections

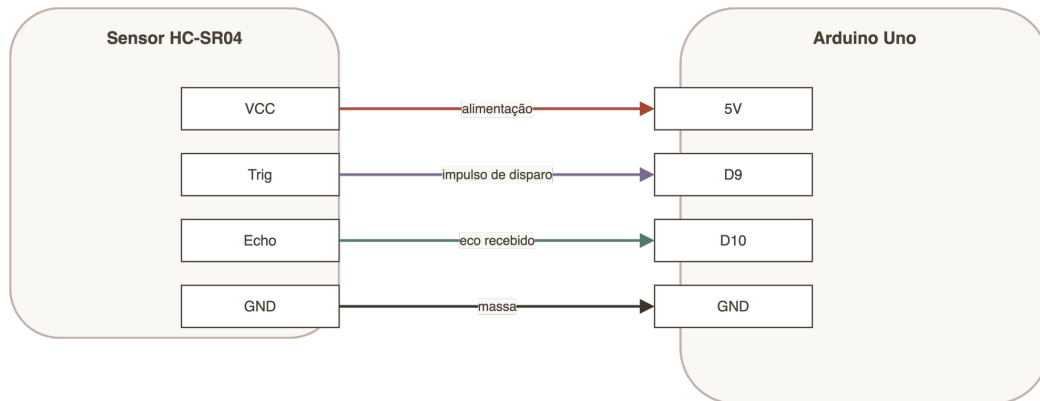


Figure 16: Connections for exercise 5

HC-SR04	Arduino	Function
VCC	5V	Power supply
Trig	D9	Trigger pulse emitted by the board
Echo	D10	Return pulse, of duration proportional to distance
GND	GND	Ground

Loading the firmware

Load the file `hcsr04_serie.ino`, supplied with this guide. The measurement requires microsecond timing, likewise incompatible with `StandardFirmata`.

```

const int trigPin = 9;
const int echoPin = 10;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {

```

```

digitalWrite(trigPin, LOW);  delayMicroseconds(2);
digitalWrite(trigPin, HIGH); delayMicroseconds(10);
digitalWrite(trigPin, LOW);

long duracao = pulseIn(echoPin, HIGH);
Serial.println(duracao * 0.034 / 2);
delay(500);
}

```

Basis of the measurement

The `pulseIn` function measures the duration of the pulse received on the *Echo* line, which corresponds to the round-trip time of the sound signal. Multiplying that duration by the speed of sound ($\approx 0,034 \text{ cm}/\mu\text{s}$) and dividing by two gives the distance to the obstacle.

Building the flow

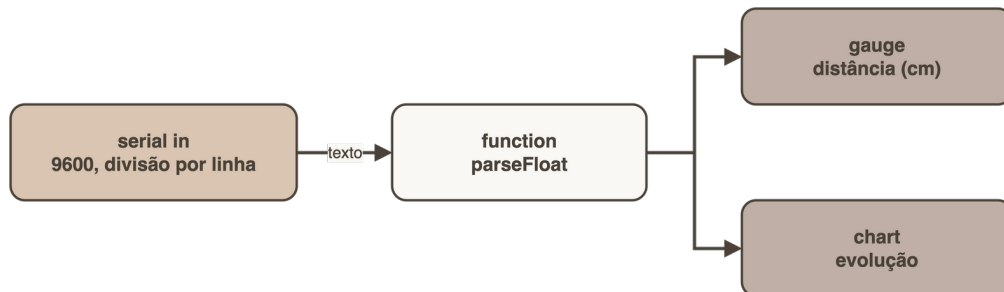


Figure 17: Node-RED flow for exercise 5

1. Drag a `serial in` node, configured as in the previous exercise.
2. Drag a `function` node containing the single line `msg.payload = parseFloat(msg.payload);` followed by `return msg;`
3. Connect a `ui-gauge` node labelled *Distance*, with range 0 to 200, and a `ui-chart` node to record the evolution over time.
4. Press **Deploy**.

Verification

Moving an obstacle in front of the sensor, the indicator should follow the variation and the chart should record the corresponding history.

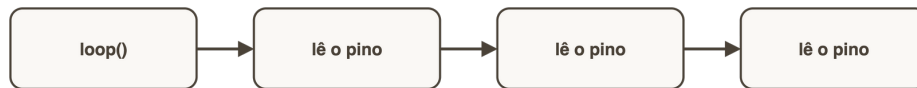
6. Interrupts at Microcontroller Level

Supplied sketches / 20 min

Objective

Establish the correspondence between the event-driven operation observed in the Node-RED flows and the interrupt mechanism implemented in the microcontroller, distinguishing it from periodic polling.

Sondagem periódica (polling)



O processador verifica repetidamente, mesmo quando não ocorre qualquer evento

Interrupção



O processador só é desviado no instante em que o evento ocorre

Figure 18: Periodic polling and interrupt

Part A — The paradigm observed in the visual flow

Before resorting to code, it is worth observing the two paradigms in Node-RED itself.



Figure 19: Periodic polling flow in Node-RED

1. Build the flow above, with an **inject** node configured to repeat every 3 seconds, a **function** node that increments a counter and a **ui-text** node on the panel.
2. Compare its behaviour with that of the flow from exercise 2. In the flow with the **inject** node, messages are emitted permanently, regardless of whether any change occurs in the environment; in the button flow, they are emitted only when a transition occurs.

Part B — Hardware interrupt

Load the *sketch interrupcao_hw.ino*, supplied with the guide. The connections are those of exercise 2, which remain unchanged.

```

const int buttonPin = 2;
const int ledPin = 13;
volatile int buttonState = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
  attachInterrupt(digitalPinToInterrupt(buttonPin), pin_ISR, CHANGE);
}
  
```



```

void loop() { /* remains empty */ }

void pin_ISR() {
  buttonState = digitalRead(buttonPin);
  digitalWrite(ledPin, buttonState);
}

```

Points to observe

The `loop()` cycle is empty: the whole behaviour resides in the `pin_ISR` routine, invoked automatically by the interrupt controller whenever pin D2 changes state. The shared variable is declared `volatile`, instructing the compiler to read it always from memory, since it may be modified outside the normal flow of execution.

Part C — Software interrupt (timer)

Load the *sketch* `interrupcao_sw.ino`. This exercise requires no additional connections.

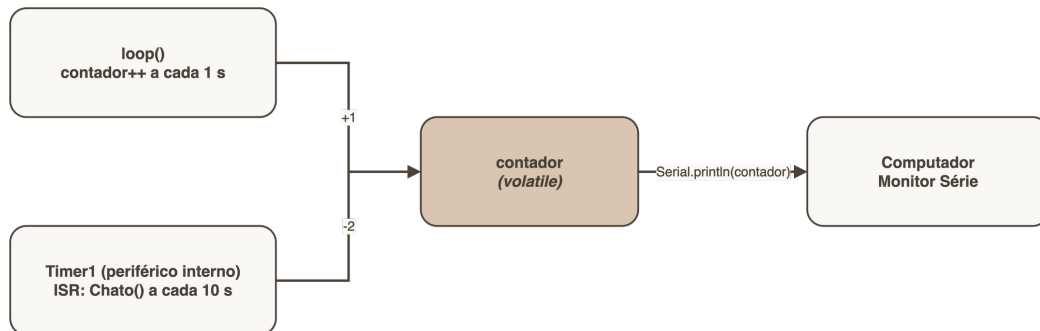


Figure 20: Two execution flows acting on the same variable

```

#include <TimerOne.h>
volatile int contador = 0;

void subtrai() {
  contador -= 2;
  Serial.print("ISR do temporizador: ");
  Serial.println(contador);
}

void setup() {
  Serial.begin(9600);
  Timer1.initialize(10000000); // 10 seconds, in microseconds
  Timer1.attachInterrupt(subtrai);
}

void loop() {
  delay(1000);
  contador++;
  Serial.print("Ciclo principal: ");
  Serial.println(contador);
}

```

Questions for discussion

1. Which of the paradigms, polling or interrupt, consumes fewer processing resources? Justify.

2. Under what circumstances might periodic polling fail to detect a short-lived event?
3. What problem would arise if the `contador` variable were not declared `volatile`?
4. Why is it recommended that an interrupt service routine be as brief as possible, and that it contain no calls to `delay()`?

7. Integrated Display Panel

Modes A and B | 10 min

Objective

Gather into a single panel the readings from the different sources used during the session, forming an elementary supervision application.

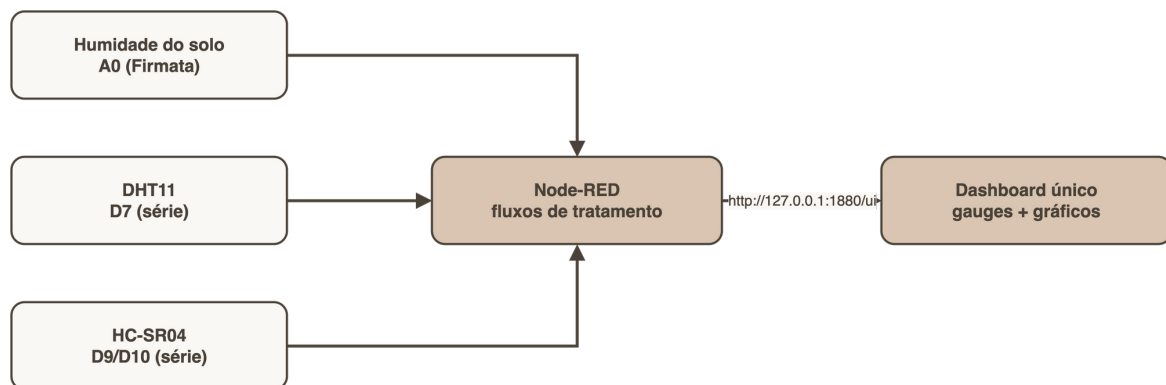


Figure 21: Architecture of the integrated panel

Procedure

1. Reload **StandardFirmata** should the analogue acquisition of exercise 3 be used simultaneously.
2. Organise the panel nodes into distinct groups, through the **dashboard** tab of the editor, giving each group a name.
3. Order the groups so as to obtain a readable layout.
4. Press **Deploy** and open <http://127.0.0.1:1880/dashboard>.

A limitation to consider

The serial port admits a single application at a time, and the board runs only one firmware at a time. Consequently, it is not possible to operate in Mode A and in Mode B on the same Arduino simultaneously. This limitation is precisely one of the motivations for the architectures studied in the following lessons, in which each device communicates autonomously through a network protocol.

Further work

Add a **switch** node from the *function* section that, when the measured distance falls below a defined threshold, drives the board LED. This yields a complete automation rule, specified exclusively by visual configuration.

References

- Node-RED documentation: <https://nodered.org/docs/>
- Arduino connection node: <https://flows.nodered.org/node/node-red-node-arduino>
- Serial communication nodes: <https://flows.nodered.org/node/node-red-node-serialport>
- Dashboard nodes: <https://dashboard.flowfuse.com>

- Firmata protocol: <https://github.com/firmata/protocol>
- External interrupts on the Arduino: <https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/>
- HC-SR04 sensor:
<https://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/>
- DHT11 sensor:
<https://www.circuitbasics.com/how-to-set-up-the-dht11-humidity-sensor-on-an-arduino/>
- TimerOne library, by Paul Stoffregen:
<https://github.com/PaulStoffregen/TimerOne>
- DHTlib library, by Rob Tillaart:
<https://github.com/RobTillaart/DHTlib>